

King Keen Chess

A full-stack multiplayer chess platform with AI opponents, gamified training, chess mini-games, and a live social layer — built in React 19 + TypeScript, powered by Supabase.

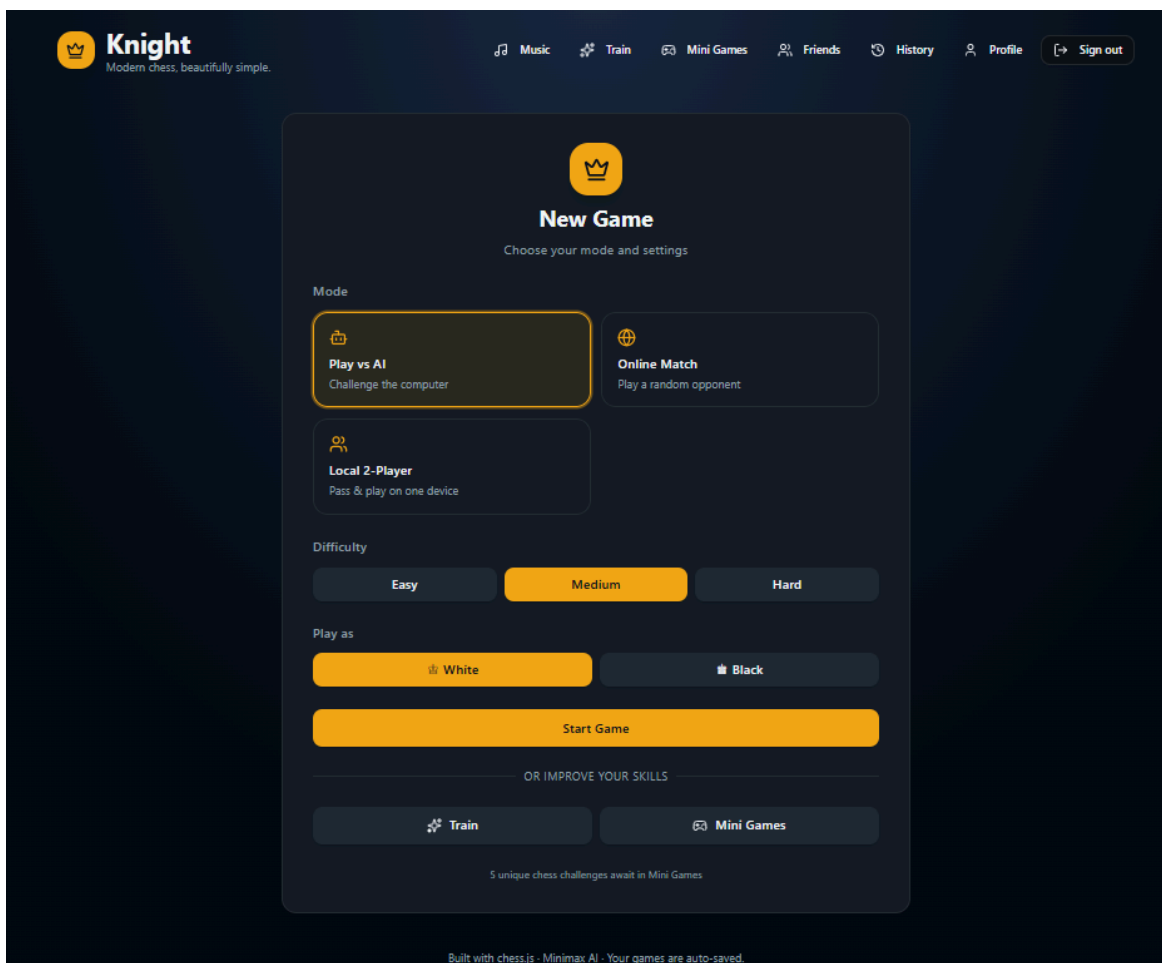


Table of Contents

- [Overview](#)
- [Tech Stack](#)
- [Features](#)
 - [Mode 1 — Standard Chess](#)
 - [Mode 2 — Training](#)
 - [Mode 3 — Mini Games](#)

- [Mode 4 — Social Layer](#)
 - [Architecture](#)
 - [Database Schema](#)
 - [Audio System](#)
 - [Getting Started](#)
 - [Environment Variables](#)
 - [Deployment](#)
 - [Project Structure](#)
-

Overview

King Keen Chess is not just a chess game — it is a full platform built around chess, with four distinct modes:

Mode	Description
Standard Chess	Local, AI, and online multiplayer with ELO, chat, draw offers, and spectating
Training	A gamified progression system with 30 levels across 6 worlds
Mini Games	Five unique chess-variant games with AI and online support
Social	Friends, presence, challenges, and spectating

The project is approximately **10,500 lines of TypeScript/TSX across 95 source files**.

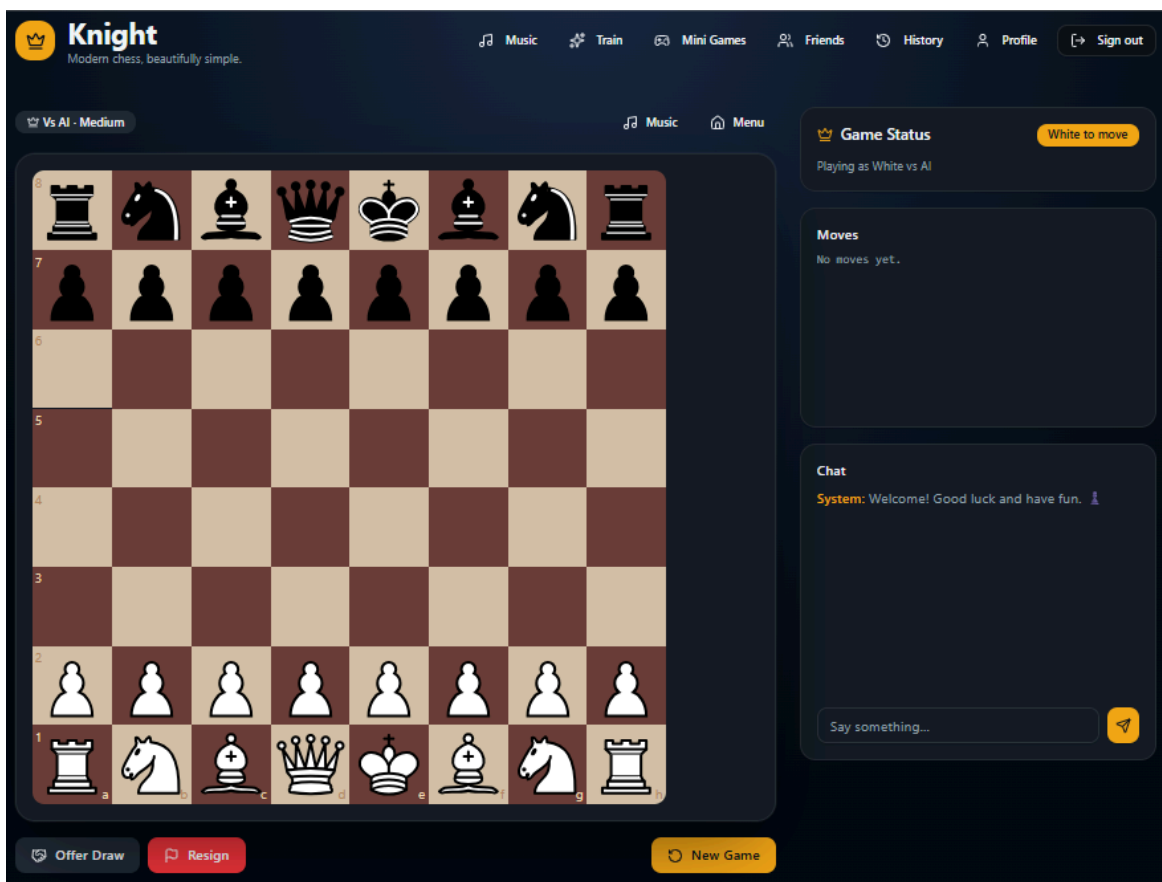
Tech Stack

Layer	Technology
Frontend	React 19, TypeScript
Routing	TanStack Router (file-based)
Data Fetching	TanStack Query
UI Components	Radix UI + shadcn/ui + Tailwind CSS v4
Backend / DB	Supabase (Postgres, Auth, Realtime, Presence)
Build Tool	Vite 7

Deployment	Vercel
Chess Logic	chess.js
Audio	Web Audio API (zero asset files — fully synthesised)

Features

Mode 1 — Standard Chess



The main game supports three sub-modes selectable from the start screen.

Local Multiplayer

Two players share the same browser. Moves are synced through a `BroadcastChannel` object — no latency, no server dependency.

AI Mode

A custom chess engine (`chess-ai.ts`) built from scratch:

- **Minimax search** with **alpha-beta pruning**

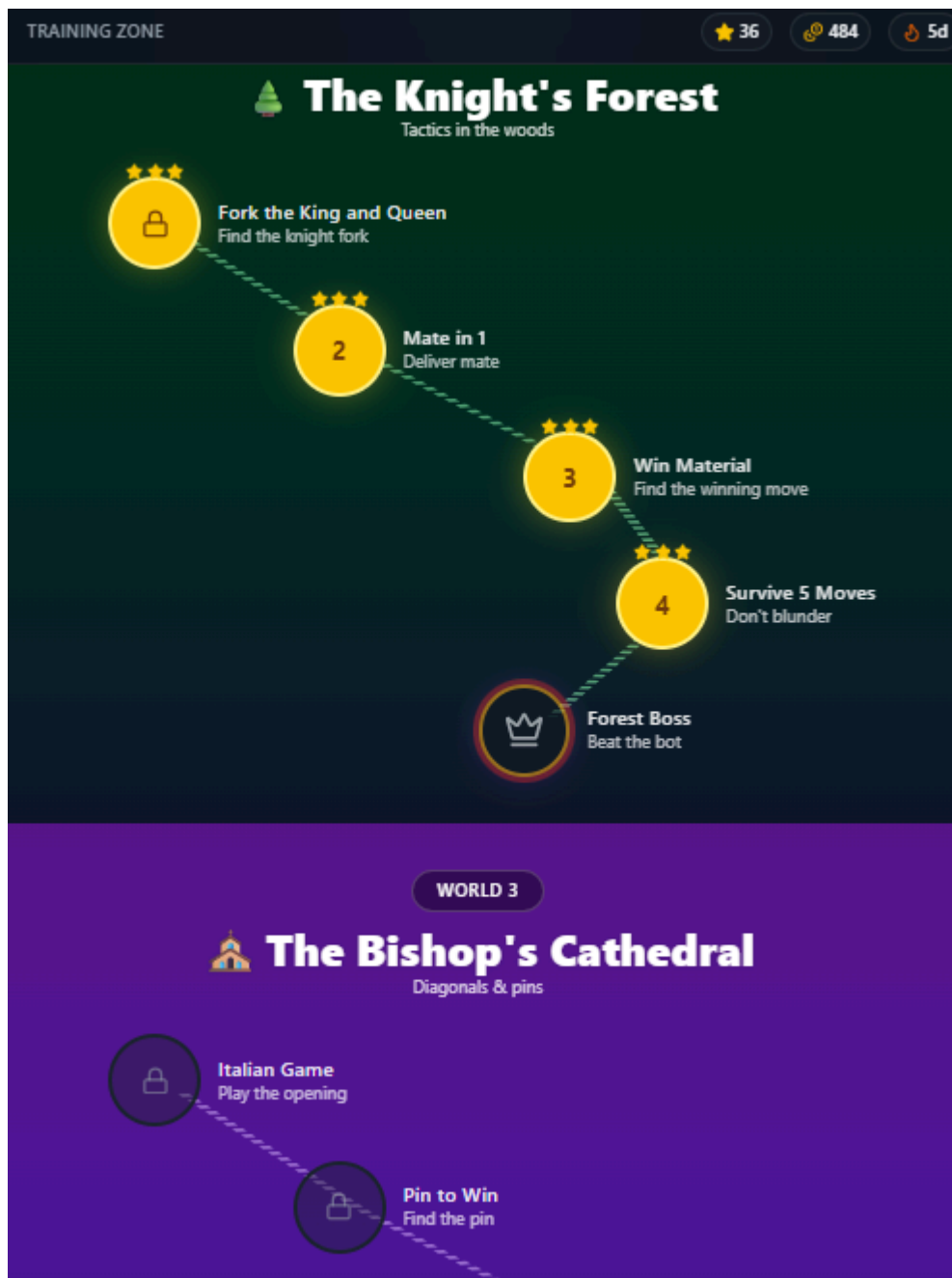
- **Move ordering** — captures are evaluated first to improve pruning efficiency
- **Piece-Square Tables (PST)** for all six piece types — 64-element arrays encoding positional bonuses (knights favour the centre, rooks target open files, kings shelter behind pawns)
- Configurable search depth (default: 3)

Online Multiplayer

- Each game gets a dedicated Supabase Realtime broadcast channel (`game-{roomId}`)
- Moves broadcast as a payload: FEN string + last move coordinates + result
- Spectators join mid-game and receive a full state re-broadcast
- `isInteractive` flag prevents out-of-turn moves
- **Matchmaking** uses atomic optimistic locking on Postgres — `.is("opponent_id", null)` prevents two players claiming the same opponent simultaneously
- Colour assignment (`w / b`) comes from the database row, not client logic

Online game features: ELO rating updates · In-game chat drawer · Draw offers · Resignation · Spectator mode · Result saved to `game_history`

Mode 2 — Training



A gamified learning curriculum with 30 levels across 6 themed worlds.

World Map

A scrollable vertical SVG map with a winding path connecting level nodes. Each world has a distinct visual theme:

World	Theme	Colour
1	The Pawn Fields	Green
2	The Knight's Forest	Dark Green

3	The Bishop's Cathedral	Purple
4	The Rook's Fortress	Grey
5	The Queen's Domain	Gold
6	The King's Court	Dark Red

Level nodes display 1–3 stars based on your best score, pulse with an animated glow when available, and show a padlock icon when locked. Scroll position is persisted in `sessionStorage` .

Progression

Level 1 of World 1 is always unlocked. Each subsequent level requires the previous to be completed. The first level of each new world requires the previous world's boss to be defeated — strict linear progression through 30 levels.

Level Types



Type	Description
------	-------------

Tutorial	Sparse board (2–4 pieces), animated SVG arrows, highlighted squares, auto-undo on wrong move
Find the Move	Given a position, find the correct move — 3 wrong attempts fail the level
Survive N Moves	Play against AI without a major blunder; health bar drops on high-value piece loss
Beat the Clock	Solve within 60 seconds; a draining timer bar shows remaining time
Opening Trainer	Play the first moves of Italian Game, Sicilian, Queen’s Gambit, or London System correctly
Boss Battle	Full game vs AI at escalating depth (scales with world number)

Guidance System

An SVG overlay (`GuidanceOverlay.tsx`) sits above the board with `pointer-events: none` . It draws animated glowing arrows, pulses highlight circles on target squares, and can spotlight a piece by dimming everything else. A lightbulb button toggles hints on/off.

Coach Character

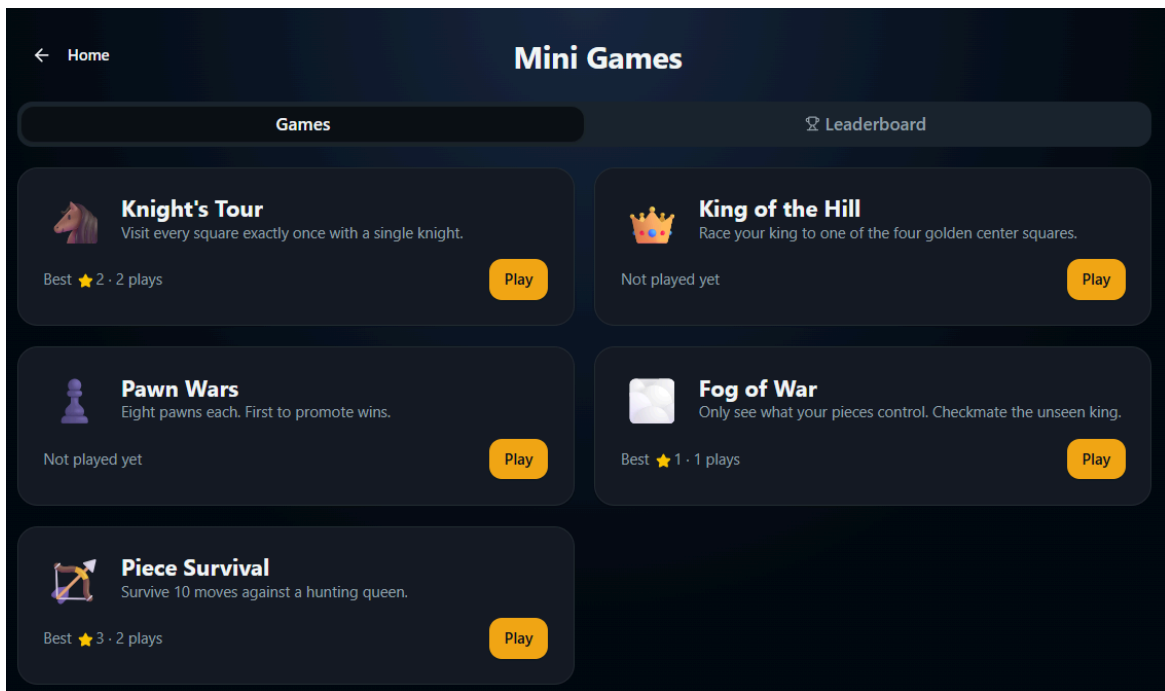
An animated SVG knight (`CoachCharacter.tsx`) with four states — idle, correct (thumbs up), wrong (shakes), celebrate — and a speech bubble with context-appropriate messages.

Game Economy

Resource	Details
Lives	Max 5, regenerate 1 every 30 minutes
Coins	Earned on level completion, scaled by stars
Stars	Cumulative across all levels
Daily Streak	Tracked per calendar day
Progress	Stored in <code>user_training_stats</code> and <code>user_level_progress</code> on Supabase

Life regeneration is calculated client-side from `last_life_lost_at` — no polling required.

Mode 3 — Mini Games



Five chess-variant games, each supporting up to three play modes: vs AI · Local multiplayer · Online multiplayer.

A `ModeSelector` component reads the game registry, greys out unavailable modes with tooltips, and remembers the last selected difficulty. An intro screen with an SVG animation previews each game before play.

Knight's Tour



A single knight must visit all 64 squares exactly once. Manages visited squares as a plain Set in React state. Valid moves computed by applying the 8 L-shape offsets and filtering for unvisited squares. In online mode, both players race on independent boards with a live opponent progress counter.

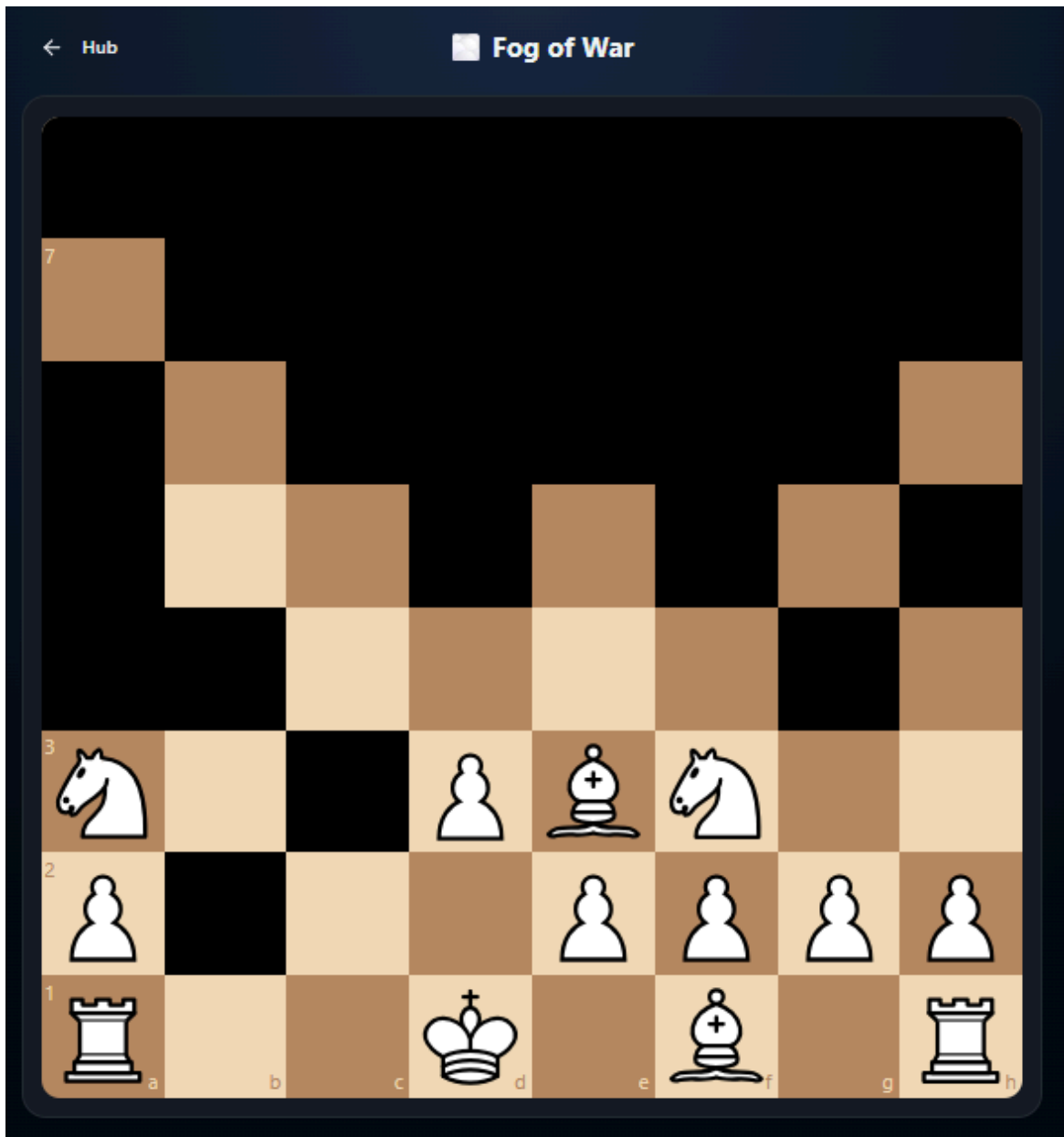
King of the Hill

Kings only — no other pieces. Move your king onto one of the four centre squares (d4, e4, d5, e5) before the opponent. The AI uses a Manhattan/Chebyshev distance heuristic to always move toward the centre — intentionally bypasses the general minimax engine for a faster greedy approach.

Pawn Wars

Eight pawns per side, no kings. Standard chess.js move validation applies, but win/loss detection is manual: win when a pawn reaches the back rank, lose when all pawns are captured. Hard mode introduces occasional sub-optimal AI moves to block your most advanced passed pawn.

Fog of War



Standard chess but the board is covered by a dark SVG overlay. `computeVision()` calculates visible squares using `chess.js`'s `isAttacked()` iterated over all 64 squares — only squares your pieces occupy or attack are visible. Easy mode extends vision one square beyond attack range. The fog calculation is **entirely client-side** — only the FEN is transmitted over the network, preventing network-traffic cheating.

Piece Survival

The survivor starts with all 16 pieces. The hunter starts with only a queen (medium) or queen + rook (hard). The hunter AI always captures the highest-value reachable piece; if no capture is available, it simulates each candidate move and picks the one that maximises pieces attacked next turn. The survivor wins if any piece survives 10 of the hunter's moves.

Online Infrastructure for Mini Games

- Private rooms use a 6-character alphanumeric code (no ambiguous characters like O / 0 or I / 1)
- Quick match uses the same atomic optimistic locking pattern as main chess matchmaking
- Role assignment for asymmetric games (e.g. Piece Survival hunter vs survivor) is stored in the room row
- Mini-game ELO tracked separately as minigame_rating on profiles (±15 points per online win/loss)

Mode 4 — Social Layer

Feature	Details
Friend Codes	Unique code per profile — find players without sharing personal info
Friend Requests	Send, accept, or reject via friend_requests table
Friendships	Accepted requests create mutual rows in friendships table
Online Presence	Live green dots via Supabase Presence (WebSocket-based)
Game Challenges	Challenge a friend directly — creates a row in game_invites
Spectating	See which friends are in active games and join as a spectator

Presence (presence.ts) broadcasts each user’s ID, username, and rating. Any connect/disconnect triggers a sync event that rebuilds the online users map for all subscribers.

Architecture

```
src/  
  routes/      # File-based routes via TanStack Router  
  components/  # Shared UI components (shadcn/Radix wrappers)  
  chess/  
    ChessGame.tsx  # Main game (762 lines)  
    chess-ai.ts    # Minimax engine with alpha-beta pruning (84 lines)
```

```
fog.ts          # Fog of War vision computation
training/
  WorldMap.tsx   # Scrollable level map
  LevelRunner.tsx # Level type dispatcher (435 lines)
  GuidanceOverlay.tsx # SVG hint arrows
  CoachCharacter.tsx # Animated knight coach
  state.ts       # useTrainingState hook (Supabase sync)
minigames/
  KnightsTour.tsx
  KingOfTheHill.tsx
  PawnWars.tsx
  FogOfWar.tsx
  PieceSurvival.tsx
  online.ts      # Shared online infrastructure
social/
  friends.tsx
  presence.ts
lib/
  client.ts      # Supabase lazy-init Proxy
  sounds.ts      # Web Audio API synthesiser
supabase/
  migrations/    # 12 ordered .sql migrations
```

The Supabase client (`client.ts`) uses a lazy-initialising `Proxy` that reads `VITE_SUPABASE_URL` and `VITE_SUPABASE_PUBLISHABLE_KEY` at runtime, supporting both client-side Vite builds and server-side rendering.

Database Schema

12 ordered migrations build and evolve the schema:

Table	Purpose
<code>profiles</code>	Auto-created on signup via Postgres trigger; stores username, rating (default 1200), mini-game rating, friend code, avatar URL
<code>game_history</code>	Win/loss/draw record per user per game

matchmaking_queue	Live pairing queue; opponent_id is the atomic claim field
friendships	Mutual friendship rows
friend_requests	Pending friend requests
game_invites	Direct game challenges between friends
training_levels	Catalogue of all 30 levels (FEN, solution UCI, type, difficulty, move limit)
user_level_progress	Per-user stars and attempt count per level
user_training_stats	Lives, coins, total stars, streak, current world/level
training_streak_days	One row per user per day trained
minigame_rooms	Online mini-game room state
minigame_queue	Mini-game matchmaking
minigame_scores	Per-game stats
minigame_personal_bests	Leaderboard data

Row Level Security is enabled on every table. Users can only read and write their own rows. Leaderboard and score tables are publicly readable. minigame_rooms allows any authenticated user to update a room they did not create (required for guests joining rooms).

Audio System

All sound effects and background music are synthesised using the **Web Audio API** — zero audio asset files.

Sound	Implementation
Move	Oscillator with triangle wave, short envelope
Capture	Higher-frequency square wave burst
Check	Sawtooth oscillator with sharp attack

Game End	Multi-oscillator chord resolution
Chat Notification	Sine wave ping
Ambient Music	5 oscillators at harmonically related frequencies, each with an independent LFO modulating its gain at a different rate — creates a slow shimmer effect on a soft A-minor chord pad; fades in/out gracefully via scheduled gain ramps

Getting Started

Prerequisites

- Node.js 18+
- A [Supabase](#) project

Installation

```
git clone https://github.com/your-username/king-keen-chess.git
cd king-keen-chess
npm install
```

Apply Database Migrations

```
npx supabase db push
```

Or run the 12 migration files in order from `supabase/migrations/` via the Supabase SQL editor.

Start Development Server

```
npm run dev
```

Environment Variables

Create a `.env.local` file in the project root:

```
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=your-anon-key
```

These are injected at build time by Vite and read at runtime by the lazy-init Supabase client.

Deployment

Platform	Config File
Vercel	<code>vercel.json</code>

Set the two environment variables (`VITE_SUPABASE_URL` , `VITE_SUPABASE_PUBLISHABLE_KEY`) in whichever platform’s dashboard you use.

Project Structure

```
king-keen-chess/
  src/
    routes/      # TanStack Router file-based routes
    components/  # Shared UI (shadcn/Radix)
    chess/       # Standard chess mode + AI engine
    training/    # Training mode + world map + coach
    minigames/   # 5 variant games + online infra
    social/      # Friends + presence
    lib/         # Supabase client, audio synthesiser
  supabase/
    migrations/  # 12 ordered SQL migrations
  public/
  vercel.json
  netlify.toml
  wrangler.jsonc
  vite.config.ts
  package.json
```
